Software Requirements Specification

Student ID	Student Name	Responsibility in the project
6622770491	Apichat Laohawan	UML, AI Model Training, API Calling
6622780037	Krit Meesiri	User Stories, User Requirement, UI interface
6622781936	Apiwit Pengyuak	Slide presentation, Presentation
6622782009	Pearanat Pipathanasiri	Slide presentation, Presentation
6622782033	Panod Manodee	BPMN, Slide presentation

Project Overview

Describe the overview of the project, its objectives and outcomes.

Email is one of the most widely used communication tools. However, users frequently receive malicious emails such as phishing attempts, scam messages, and unwanted subscription notifications. Many users cannot easily distinguish between legitimate and harmful emails, which increases the risk of financial loss, data leakage, or fraud.

This project proposes a browser-based solution that assists users in analyzing emails automatically when they are opened in Gmail.

This document is organized as follows:

Section 1 gives a global introduction to the document and the software product.
Section 2 informally introduces the software product and the context where it will be used.
Section 3 specifies the requirements for the software product.

Revision History

Specifies details of the revision version of the documents.
Recommendation:

- Use version no. 0.1, 0.2, … 0.x for the in-progress versions of the document.
- Use version no. 1.0 for the final and approved version.

Version	Date	Author	Change Description
0.1	30 Jan 2026	Krit, Panod, Pearanat, Apiwit, Apichat	Draft and Revised scorp
0.2	1 Feb 2026	Krit, Panod, Apichat	Add UML
0.3	5 Feb 2026	Apichat, Pearanat	Add BPMN
0.6	10 Feb 2026	Panod, Krit	Draft UI and Use Case
0.8	15 Feb 2026	Apiwit	Appendix
1.0	10 Apr 2026	Apichat	Training AI, API Calling
1.1	16 Apr 2026	Krit	UI/UX and Testing, Debugging
1.2	25 Apr 2026	Krit, Panod, Pearanat, Apiwit, Apichat	Update UI/UX and Testing, Debugging

TABLE OF CONTENTS

- INTRODUCTION
  - Purpose
  - Scope
  - Overview
  - Definitions, Acronyms, and Abbreviations
  - References
- General Description
  - Product Perspective
  - Product Functions
  - User Characteristics
  - General Constraints
  - Assumptions and Dependencies
- Requirements
  - Functional Requirements
  - External Interface Requirements
    - User Interfaces
    - Hardware Interfaces
    - Software Interfaces
    - Communications Interfaces
  - Performance Requirements
  - Design Constraints
    - Standards Compliance
    - Hardware Limitations
  - Attributes
    - Availability
    - Security
    - Maintainability
    - Transferability/Conversion
  - Other Requirements
    - Operations
    - Site Adaptation
- Appendixes
  - Annex 1: Characteristics of a good SRS
  - Annex 2: Organizing the specific requirements

Annex 3: SRS templates

Template of SRS Section 3 organized by feature

Template of SRS Section 3 organized by user class

Template of SRS Section 3 showing multiple organizations

Section 1. INTRODUCTION

Purpose of this document

This Software Requirements Specification (SRS) document provides a comprehensive description of the Email Detection system. It presents an overview of the system's objectives, operating environment, features, constraints, and detailed software requirements. The document serves as a structured reference for understanding how the system is intended to function and what capabilities it must provide.

The primary purpose of this SRS is to clearly define and formalize the functional and non-functional requirements of the software product. It specifies the system's expected behavior, including inputs, processing, and outputs, in a manner that supports system design, implementation, and testing. This document ensures that all requirements are well-defined, testable, and externally observable.

The intended audience of this SRS includes system developers, project supervisors, testers, and other stakeholders involved in the design, development, evaluation, and maintenance of the Email Detection system. It may also serve as a reference document for academic assessment and future system enhancements.

Scope of the Software System

The Email Detection system is a browser-based Chrome Extension designed to assist Gmail users in identifying potentially malicious or subscription-related emails. The system operates within the Gmail web interface and analyzes the visible content of opened emails using an integrated AI model trained by the developer.

The software product will detect when a user opens an email, extract the subject and body text, and classify the email into one of three categories: safe, malicious, or subscription-related. Based on the classification result, the system will display an appropriate notification to inform the user. In the case of subscription-related emails, the system will provide guidance to help the user understand possible cancellation or management steps.

The system will not automatically delete emails, modify email content, access attachments, or perform subscription cancellation on behalf of the user. All final decisions and actions remain under user control.

The primary objective of the software is to enhance user awareness and decision-making when interacting with email content. The system aims to reduce the risk of phishing and scam attacks, improve user understanding of subscription-related messages, and provide a lightweight, privacy-focused AI assistance tool that operates directly within the browser environment.

The scope defined in this document is consistent with the overall project objective, which focuses on software-based email analysis and user notification without external system integration or automated account-level control.

Definitions, Acronyms, and Abbreviations

Term/Acronym	Definition
AI	A computational model or system capable of analyzing data and making classification or prediction decisions based on learned patterns.
Chrome Extension	A browser-based software component that extends the functionality of the Google Chrome web browser.

Email Detection System	**The software product described in this document, designed to classify and provide notifications regarding email content within Gmail.**
IEEE	**Institute of Electrical and Electronics Engineers.**
MRD	**Marketing Requirements Document.**
Phi-3-mini	**A lightweight language model integrated into the system for analyzing and classifying email content.**
Phishing	**A type of cyberattack where attackers attempt to trick users into revealing sensitive information.**
Ransomware	**A type of malicious attack that demands payment by threatening data loss or exposure.**
Subscription-related Email	**An email associated with recurring services, billing notifications, or subscription confirmations that may require user management.**
Malicious Email	**An email that may contain phishing attempts, scams, or deceptive content intended to mislead users.**

References

https://developers.google.com/workspace/gmail/api/guides

https://developer.chrome.com/docs/extensions

IEEE Standard Glossary of Software Engineering Terminology," in ANSI/ IEEE Std 729-1983 , vol., no., pp.1-40, 18 Feb. 1983, doi: 10.1109/IEEESTD.1983.7435207.

Section 2. General Description

This section provides an overview of the Email Detection system and describes the general factors that influence its design and requirements. It introduces the operating context, system characteristics, and usage environment in order to provide background information for the detailed requirements specified in Section 3.

The Email Detection system is developed as a browser-based Chrome Extension that operates within the Gmail web interface. The system is designed to assist users in identifying potentially malicious or subscription-related emails through AI-based classification. It functions as a lightweight support tool that enhances user awareness rather than replacing user decision-making.

The system operates entirely within the user's browser environment. It analyzes visible email content when an email is opened and generates a classification result using an integrated AI model trained by the developer. The system does not modify email data, access attachments, or perform automatic actions such as deleting emails or canceling subscriptions. All final decisions remain under user control.

The primary users of the system are general Gmail users, including students and everyday internet users who may not have advanced technical knowledge of cybersecurity. Therefore, the system interface and notifications are designed to be simple, clear, and easily understandable.

The operating environment consists of the Google Chrome browser and the Gmail web platform. The system does not require a dedicated backend server or external integration for its core functionality. Privacy considerations play an important role in the system design, as the software avoids permanent storage of email content.

Overall, this section provides contextual background to support a clear understanding of the detailed functional and non-functional requirements presented in the following section.

System Perspective and Environment

The Email Detection system is a browser-based application implemented as a Chrome Extension. It operates within the Gmail web interface and functions as an assistive tool that enhances email content awareness. The system is not a stand-alone web application, mobile application, or IoT-based system. Instead, it runs locally inside the user's web browser and integrates directly with the Gmail interface.

The system is developed using web-based technologies, including JavaScript, HTML, and CSS for the extension interface and interaction with the Gmail webpage. The AI classification component is based on a locally integrated language model trained by the developer. No centralized cloud backend or external database is required for core functionality.

The system interacts with the Gmail web interface by accessing the Document Object Model (DOM) to detect when an email is opened and extract visible subject and body content. It does not require external authentication APIs or third-party service integration. No specialized hardware interfaces are required beyond a standard personal computer capable of running Google Chrome.

The deployment environment consists of:

- *Google Chrome browser*
- *Gmail web platform*
- *A local user computing environment (desktop or laptop)*

The system is installed as a Chrome Extension and operates entirely within the browser environment without requiring additional infrastructure.

System Features

The Email Detection system provides AI-assisted analysis of email content within the Gmail web interface. The primary function of the system is to support users in identifying potentially harmful or subscription-related emails through automated classification and notification.

When a user opens an email, the system detects the email view and extracts the visible subject and body content. The extracted content is analyzed using an integrated AI model trained by the developer. Based on this analysis, the system classifies the email into one of three categories: safe, malicious, or subscription-related.

If the email is identified as malicious, the system displays a warning notification to alert the user of potential phishing or scam risks. If the email is identified as subscription-related, the system provides guidance suggesting possible subscription management or cancellation steps. If the email is classified as safe, a safe status indicator is shown.

The system is designed to assist users in decision-making without performing automatic actions. It does not modify email content, delete messages, access attachments, or cancel subscriptions. All actions remain under the user's control.

Overall, the system functions as an intelligent notification layer integrated within Gmail to enhance user awareness and email security.

Stakeholders and users

Stakeholders

The following stakeholders are involved in this project:

1. Developer
The developer is responsible for designing, implementing, training the AI model, integrating the model into the Chrome Extension, and maintaining the system. The developer ensures that the system functions correctly and meets the specified requirements.

2. End Users (Gmail Users)
End users are individuals who install and use the Email Detection Chrome Extension. They interact directly with the system and receive classification results and notifications.

3. Project Supervisor / Instructor
The project supervisor evaluates the system design, implementation, and documentation. The supervisor ensures that the project meets academic and technical standards.

Intended User Characteristics

The primary users of the system are general Gmail users, including students and everyday internet users.

Key characteristics include:

- ***Age Range:** Typically 18–40 years old (university students and working adults), though the system can be used by any Gmail user.*
- ***Educational Level:** Secondary education or higher.*
- ***Technical Experience:** Basic computer literacy, including the ability to use a web browser and Gmail.*
- ***Cybersecurity Knowledge:** Users are not expected to have advanced knowledge of phishing detection or AI systems.*

The system is designed with simplicity in mind to accommodate users with limited technical expertise. Notifications and explanations are presented in clear and understandable language to ensure accessibility.

General Constraints

The development of the Email Detection system is subject to several general constraints that influence implementation decisions.

a) Regulatory policies;

The system must comply with browser extension policies and Gmail platform usage policies. Since the software operates within the Google Chrome browser and interacts with the Gmail web interface, it must follow Chrome Extension development guidelines and avoid unauthorized data collection or misuse of user information.

b) Hardware limitations (e.g., signal timing requirements);

The system operates entirely within a user's personal computer through the Chrome browser. Performance may depend on the user's device capabilities, including CPU performance and available memory, as the AI model runs locally. No specialized hardware is required.

c) Interfaces to other applications;

The system interfaces directly with the Gmail web interface through browser-based interaction (DOM access). It does not integrate with external third-party systems, APIs, or backend services. Therefore, its functionality depends on the structure and availability of the Gmail web interface.

i) Reliability requirements;

The system shall operate reliably within the Google Chrome browser environment. The extension shall not interrupt or interfere with the normal functionality of Gmail. If AI classification fails or encounters an error, the system shall display a neutral message rather than producing incorrect or misleading output.

j) Criticality of the application;

The Email Detection system is a supportive tool and is not classified as a safety-critical or mission-critical system. Incorrect classification may affect user perception, but it does not directly cause financial transactions, account modification, or system-level changes. Final decisions remain under user control.

k) Safety and security considerations.

The system must ensure user privacy and data protection. Email content shall not be permanently stored or transmitted to unauthorized external services. All analysis shall be performed locally within the browser environment. The system shall not modify, delete, or automatically act on email content.

Security considerations focus on minimizing data exposure and preventing misuse of user information.

Assumptions and Dependencies

This section describes the assumptions and external factors that may affect the requirements of the Email Detection system. These are not design constraints, but changes to these factors may impact the system's functionality.

Assumptions

- *It is assumed that users access Gmail through the Google Chrome browser.*
- *It is assumed that users have a stable internet connection when accessing Gmail.*
- *It is assumed that the Gmail web interface structure remains consistent and accessible for content extraction.*
- *It is assumed that users open emails normally within the Gmail web interface (not through external mail clients).*

Dependencies

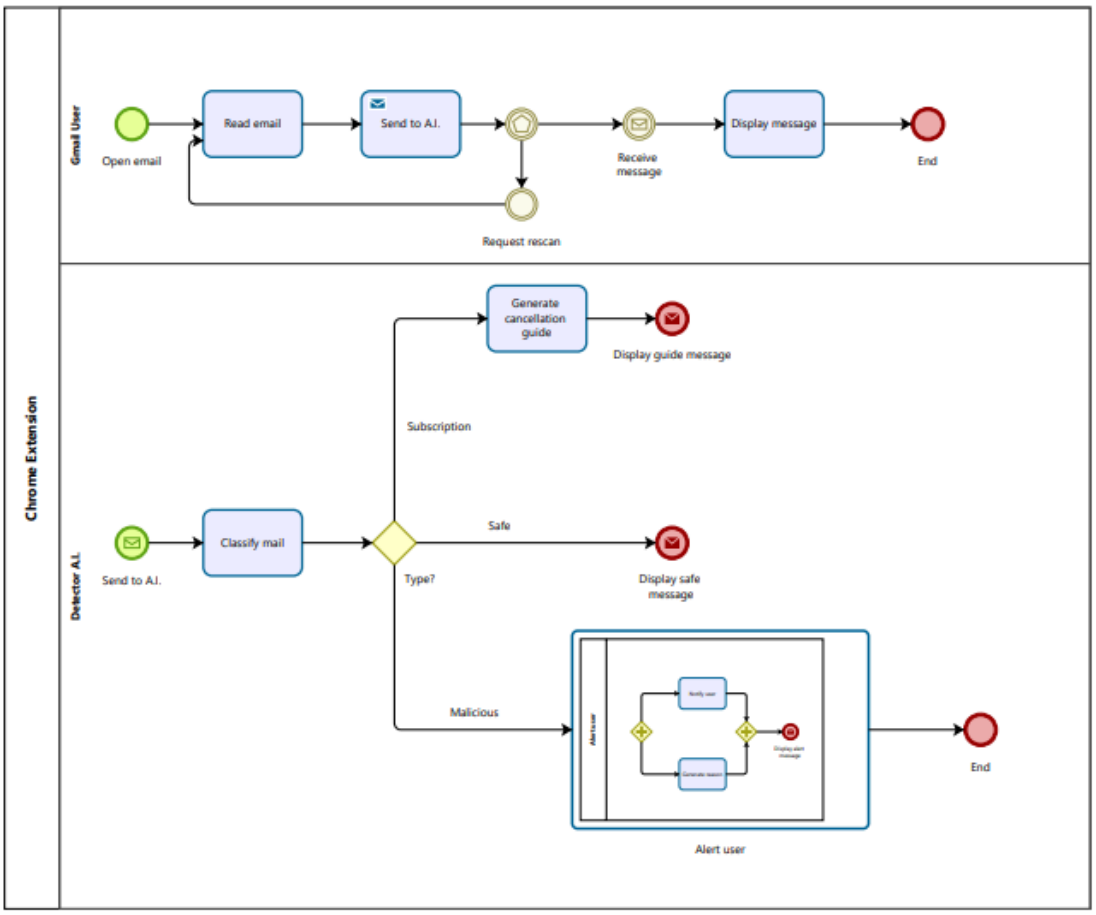
- *The system depends on the Google Chrome browser environment to operate as a Chrome Extension.*

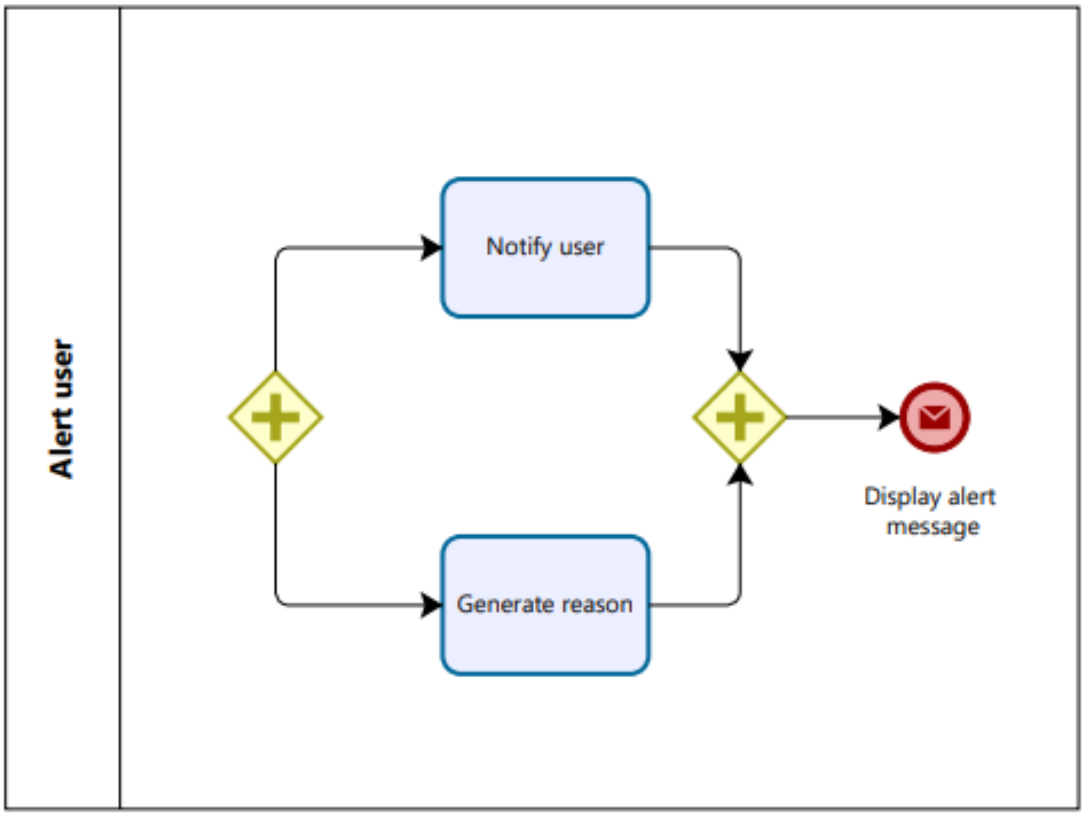
- *The system depends on the Gmail web interface structure (DOM layout) to detect opened emails and extract content. Significant changes to Gmail's interface may require system updates.*

- *The system depends on the integrated AI model functioning correctly within the local environment. Model performance may affect classification accuracy.*

Section 3. Requirements

Business Process





This section specifies the functional and non-functional requirements of the Email Detection system. All requirements described in this section are externally observable and testable within the Gmail web interface.

Conformance to Requirement Characteristics

All requirements in this section are written to be:

- ***Correct*** *– They reflect the intended functionality of the Email Detection Chrome Extension.*
- ***Unambiguous*** *– Each requirement clearly describes system behavior.*
- ***Complete*** *– All major system functions (detection, extraction, classification, notification) are included.*
- ***Consistent*** *– Requirements do not conflict with one another.*
- ***Verifiable*** *– Each requirement can be tested within the Gmail interface.*
- ***Modifiable and Traceable*** *– Each requirement is uniquely identified for future updates and reference.*

Cross-Reference to Earlier Sections

The requirements specified in this section are consistent with:

- *The system scope defined in Section 1, which states that the software operates as a Chrome Extension within Gmail.*

- *The system features described in Section 2, which outline email detection, AI classification, and user notification.*
- *The constraints and assumptions identified earlier, including operation within the Chrome browser and reliance on the Gmail web interface.*

Unique Identification of Requirements

All requirements are uniquely identified using structured identifiers (e.g., REQ-01, REQ-02, etc.). This ensures traceability between user stories, system design, implementation, and testing activities.

Organization and Readability

Requirements are organized into functional and non-functional categories to improve clarity and readability. Functional requirements describe the system's observable behavior in response to user actions (e.g., opening an email), while non-functional requirements describe performance, reliability, privacy, and compatibility expectations.

This structured organization supports efficient system design, implementation, and validation.

Functional Requirements

User Stories

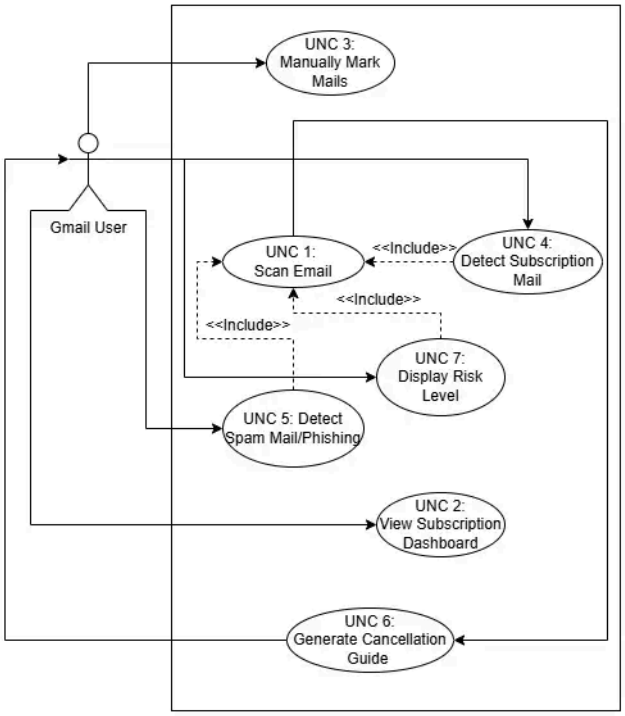
Project Name:	Maily Detector			
Recorded by:	Group10			
User Story ID	As (role of user)	I want...	for	Priority (1:low - 5: high)
USR001	Gmail User	I want the system to analyze an email when I open it,	so that I can know whether the email is safe or risky.	5
USR002	Gmail User	I want malicious or phishing emails to be identified,	so that I can avoid scams and security threats.	5
USR003	Gmail User	I want to see a clear classification result (Safe / Malicious / Sub	so that I can quickly understand the email type.	4
USR004	Gmail User	I want subscription-related emails to be detected,	so that I can track services I have subscribed to.	5
USR005	Gmail User	I want to receive AI-generated cancellation guidance for subsc	so that I can manually cancel unwanted services.	5
USR006	Gmail User	I want the system to provide a short explanation for malicious	so that I understand the reason for the warning.	4
USR007	Gmail User	I want my email content to be processed securely and not perr	so that my privacy is protected.	5

User Requirements

No.	Role of User	Name of Product Owner	User Requirements (User Stories)	Functional Requirement
1	Gmail User	Mr.Will	USR001: want to analyze an email when opening it	FR001: The system shall analyze an email when the user opens the email in the Gmail web interface.
				FR002: The system shall extract visible subject and body content from the email page.
2	Gmail User	Ms.Ann	USR002: want malicious or phishing emails to be detected	FR003: The system shall classify the email as Safe, Malicious, or Subscription.
				FR004: If classified as Malicious, the system shall display a warning notification.
3	Gmail User	Mr.Nick	USR003: want subscription-related emails to be detected	FR005: If the email is classified as Subscription-related, the system shall display a subscription notification.
				FR006: The system shall generate AI-based cancellation guidance.
4	Gmail User	Ms.Beam	USR004: want email data to be securely processed	FR007: The system shall process email content locally within the browser environment.
				FR008: The system shall not permanently store email content.
				FR009: The system shall not transmit email content to unauthorized external servers.

System Requirements		
Quality Requirement	Use Case	Use Case Description
QR001: Email classification response time must be within 3 seconds.	UC001	UD001 – Analyze Email
QR002: The AI model must achieve at least 80% classification accuracy.		
QR003: False positive rate should be minimized.	UC002	UD002 – Threat Detection Process
QR004: Warning messages must be clear and understandable.		
QR005: Subscription detection accuracy must be acceptable (at least 80%).	UC003	UD003 – Subscription Detection & Guidance
QR006: Cancellation guidance must be written in simple and non-technical language.		
QR007: The system shall comply with browser security policies.	UC004	UD004 – Secure Email Processing
QR008: The system must protect user privacy at all times.		

UML User Case Diagram



Use Case Descriptions

Maily Detector: Scan Email		
Use Case ID	UC01	
Relationship	-	
Objective	To allow the user to manually scan an opened email and receive a risk score with explanation.	
Primary Actor(s)	Gmail User	
Secondary Actor(s)	Mail Detector System	
Input data	-Email subject -Email body content -Links contained in email	
Output data	-Risk score (0–100) -Risk level (Safe / Suspicious / Dangerous) -Explanation of classification -Recommendation message (e.g., delete, unsubscribe, ignore)	
Trigger	User clicks the "Scan" button.	
Pre-condition (if any)	-User is logged into Gmail. -User has opened an email. -Extension is active.	
Post-condition (if any)	-Risk score is displayed. -Explanation is shown. -Recommendation is provided.	
	Actor	System
Normal Operation Flow	1. User opens an email. 2. User clicks "Scan" button.	3. System extracts email content. 4. System analyzes content using AI model. 5. System calculates risk score (0–100). 6. System generates explanation of detected patterns. 7. System displays risk score and explanation. 8. If spam/phishing detected → display recommendation to delete or avoid clicking links.
		9. If subscription detected → display unsubscribe guidance. 10. If no risk detected → display safe message.
Exceptional Flow	-	-

Maily Detector: Re-Scan Email		
Use Case ID	UC02	
Relationship	<<extend>> UC01 Scan Email	
Objective	To allow the user to scan a newly opened email manually.	
Primary Actor(s)	Gmail User	
Secondary Actor(s)	Mail Detector System	
Input data	-Email subject -Email body	
Output data	- Updated risk score - Updated explanation	
Trigger	User clicks "Scan" button again.	
Pre-condition (if any)	- User has opened another email.	
Post-condition (if any)	- New email analysis result is displayed.	
	Actor	System
Normal Operation Flow	1. User opens another email. 2. User clicks "Scan" again.	3. System extracts new email content. 4. System performs analysis. 5. System displays updated score and explanation.
Exceptional Flow	-	If extraction fails → display "Unable to analyze this email."

Maily Detector: View Risk Details	
Use Case ID	UC03
Relationship	<<extend>> UC01
Objective	To allow the user to view detailed explanation of the risk score.
Primary Actor(s)	Gmail User
Secondary Actor(s)	
Input data	-Previously generated risk result
Output data	- Detailed risk breakdown - Suspicious elements highlighted
Trigger	User clicks "View Details".
Pre-condition (if any)	- User has opened another email.
Post-condition (if any)	- New email analysis result is displayed.

	Actor	System
Normal Operation Flow	1. User clicks "View Details".	2. System retrieves explanation data. 3. System displays detailed risk factors.
Exceptional Flow	-	-

Maily Detector: Detect Spam/Phishing		
Use Case ID	UC04	
Relationship	Detect Spam/Phishing	
Objective	To detect whether an email contains spam or phishing patterns.	
Primary Actor(s)	System	
Secondary Actor(s)	Email content	
Input data	-Spam detection result -Suspicious indicators	
Output data	- Updated risk score - Updated explanation	
Trigger	Triggered during Scan Email.	
Pre-condition (if any)	- User has opened another email.	
Post-condition (if any)	- New email analysis result is displayed.	
	Actor	System
Normal Operation Flow		1. System analyzes suspicious keywords. 2. System checks link integrity. 3. System assigns spam risk score.
Exceptional Flow	-	-

Maily Detector: Detect Subscription Email		
Use Case ID	UC05	
Relationship	<<include>> UC01	
Objective	To detect subscription-related email patterns.	
Primary Actor(s)	System	
Secondary Actor(s)		
Input data		
Output data		
Trigger		
Pre-condition (if any)		
Post-condition (if any)		
	Actor	System
Normal Operation Flow		1. System detects billing/receipt keywords. 2. System identifies recurring payment patterns. 3. System flags as subscription-related if criteria met.
Exceptional Flow	-	-

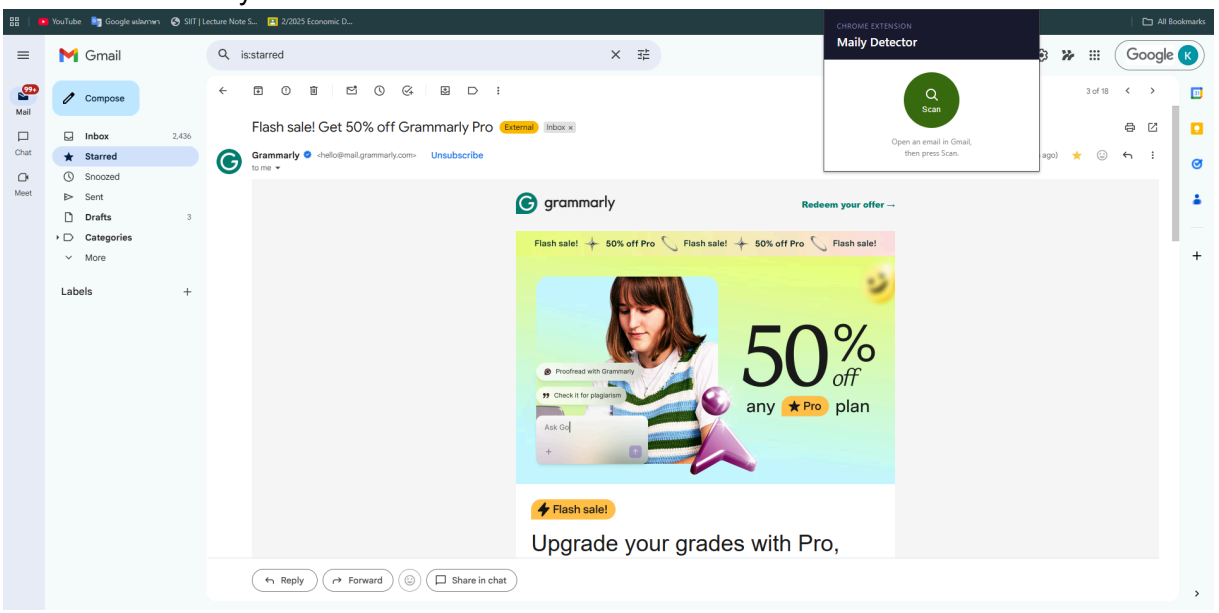
Maily Detector: Generate Risk Explanation		
Use Case ID	UC06	
Relationship	<<include>> UC01	
Objective	To generate explanation for classification result.	
Primary Actor(s)		
Secondary Actor(s)		
Input data		
Output data		
Trigger		
Pre-condition (if any)		
Post-condition (if any)		
	Actor	System
Normal Operation Flow		1. System identifies triggered risk indicators. 2. System generates explanation text.
		3. Explanation is attached to scan result.
Exceptional Flow	-	-

External Interface Requirements

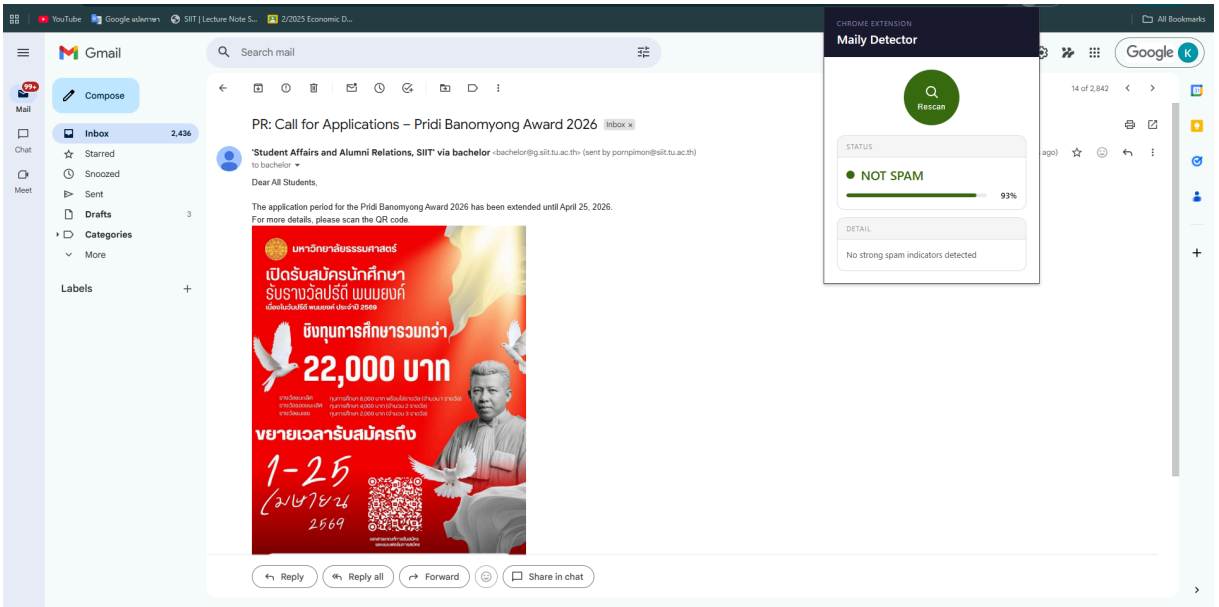
User Interfaces

User Interfaces:

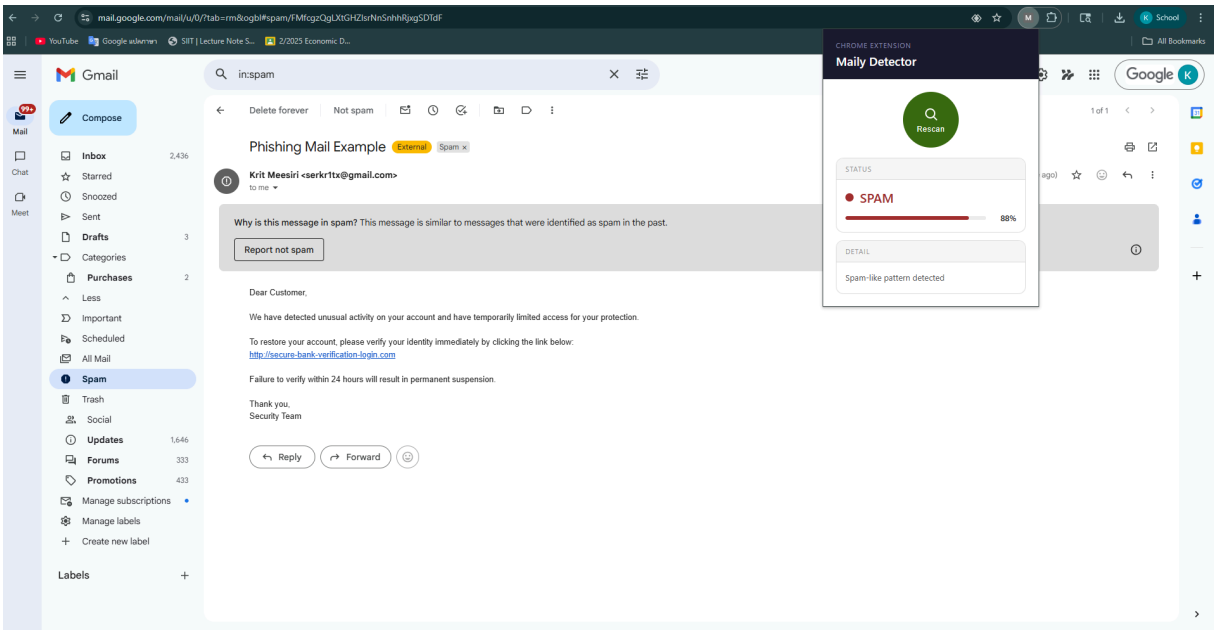
- On stand by



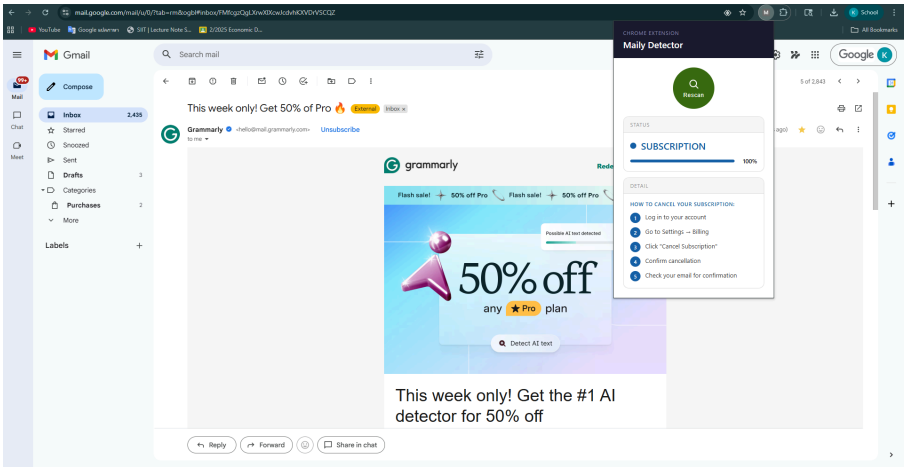
- Scan Email:
  - Safe



- Unsafe



- Subscription



Software Interfaces

The system interfaces with:

- *Google Chrome browser environment*
- *Gmail web interface (via DOM access)*
- *Hugging Face Spaces API (for spam classification model inference)*

The AI model is deployed on Hugging Face Spaces and accessed via HTTP POST requests to the Gradio API endpoint. No local model inference is performed within the extension.

The AI model is integrated locally within the extension and does not require external API calls.

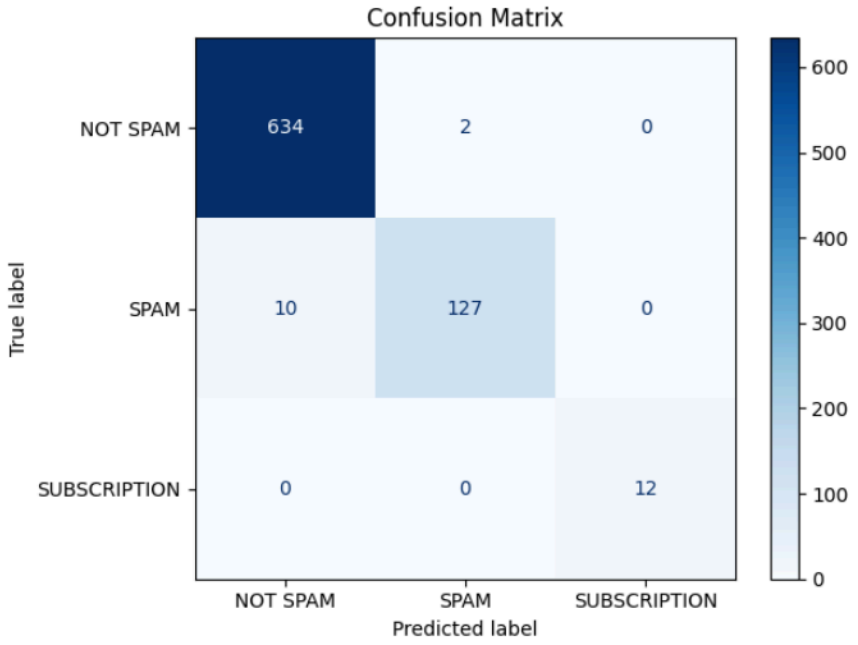
This is the result and some of the output of this model.

1. Confusion matrix:

```
                precision    recall  f1-score   support

      NOT SPAM       0.98      1.00      0.99       636
          SPAM       0.98      0.93      0.95       137
  SUBSCRIPTION       1.00      1.00      1.00        12

      accuracy                           0.98       785
     macro avg       0.99      0.97      0.98       785
  weighted avg       0.98      0.98      0.98       785
```




```
Confusion matrix saved as confusion_matrix.png
```

The Confusion matrix show that the model accuracy for all three categories is pretty accurate
The score are as follow:

The score for “Not Spam” which refer to normal mail: 99%
Only 2 emails were misclassified as Spam mail.

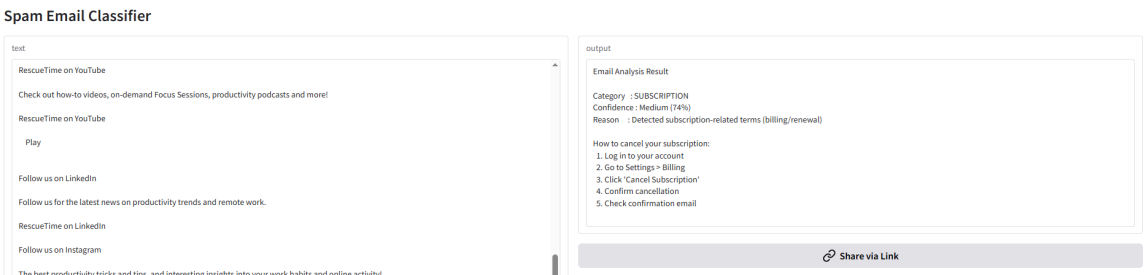
The score for “Spam” which refer to possible phishing email or other unsafe email: 95%
Only 15 emails were misclassified as normal mail.

The score for “Subscription” which refer to Subscription email: 100%
With no misclassified email.

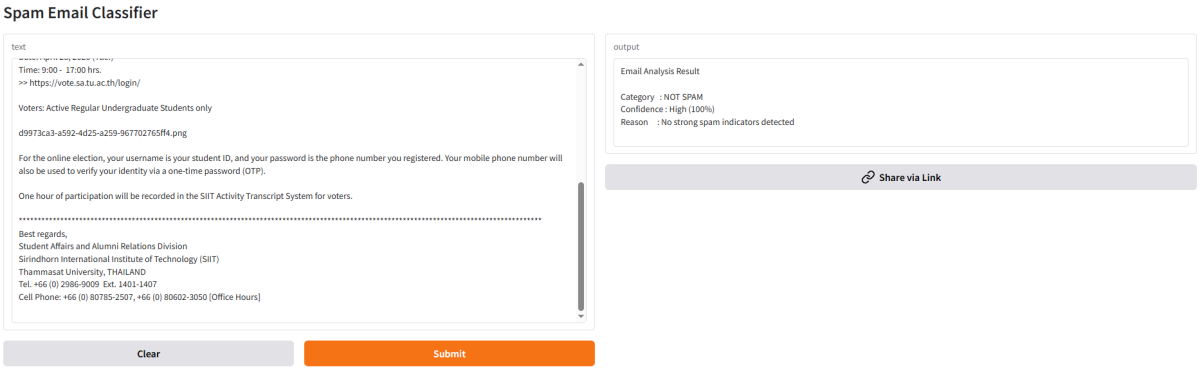
The Overall accuracy for this model is 98% from 785 emails
Only 12 emails are misclassified.

2.Example output:

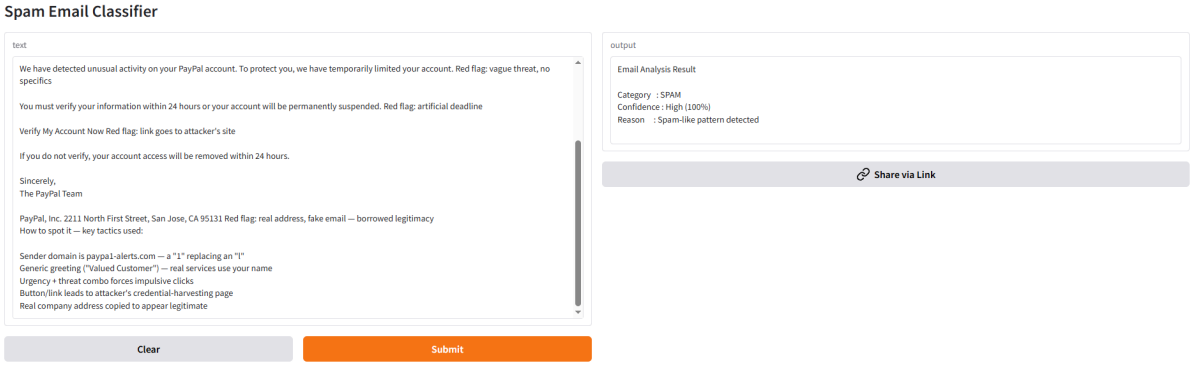
1) This is the the output of Rescuetimes Subscription Email:



2) This is the output of Siit Announcement Email:



3) This is the output of mock up Phishing Email:



The Email Detection system does not use custom communication protocols, local network protocols, or server-to-server communication. The system operates locally within the Google Chrome browser environment.

However, the system depends on the user's internet connection to access the Gmail web interface. The communication between the browser and Gmail servers is handled by the standard HTTPS protocol managed by the browser. The extension itself does not implement or manage communication protocols directly.

No additional communication interfaces such as TCP/IP socket communication, message queues, or external API services are used.

Performance Requirements

The system shall meet the following measurable performance requirements:

Static Requirements

- *The system shall support one active user session per browser instance.*

- *The system shall process one opened email at a time.*
- *The system shall handle email subject and body text up to 50,000 characters.*

Dynamic Requirements

- *The system shall display a classification result within **3 seconds** after an email is opened under normal operating conditions.*
- *At least **95% of email classifications** shall be completed within 3 seconds on a standard personal computer.*
- *The system shall not increase Gmail page load time by more than **10%** under normal usage conditions.*
- *If classification fails, the system shall return an error or neutral message within **3 seconds**.*

All performance measurements assume a standard desktop or laptop environment with stable internet access.

Design Constraints

The system is subject to the following design constraints:

- *The system must be implemented as a Google Chrome Extension.*
- *The system must operate within the Gmail web interface.*
- *The system must be developed using web technologies (JavaScript, HTML, CSS).*
- *The system must not require a backend server or external cloud infrastructure.*
- *The system must process email data locally and must not permanently store user email content.*
- *The system must comply with Chrome Extension platform policies.*

No specialized hardware devices or external enterprise systems are required.

Standards Compliance

The Email Detection system complies with applicable software development and platform standards relevant to its operating environment.

a) Report Format

This Software Requirements Specification (SRS) document follows a structured format based on IEEE recommendations for software requirements documentation. Requirements are uniquely identified and organized to ensure clarity, traceability, and testability.

b) Coding and Data Standards

The system shall be developed using standard web technologies, including JavaScript, HTML, and CSS, following widely accepted coding conventions for readability and

maintainability.

The system shall comply with:

- *Google Chrome Extension development guidelines.*
- *Secure handling of user data within browser environments.*
- *Standard HTTPS communication protocols handled by the browser.*

No proprietary data format or external database standard is required, as the system does not use a backend database.

c) Software Quality Assurance (SQA) Process

The system shall undergo:

- *Functional testing to verify correct email classification behavior.*
- *Performance testing to ensure classification results are returned within specified time limits.*
- *Manual testing within the Gmail interface to verify UI integration and notification display.*
- *Code review to ensure maintainability and adherence to design constraints.*

The system does not require regulatory audit trace functionality, financial logging compliance, or government certification processes.

Attributes

The following software attributes are required and shall be objectively verifiable:

Performance

- *At least 95% of email classifications shall complete within 3 seconds under normal operating conditions.*
- *The system shall not noticeably degrade Gmail interface responsiveness.*

Reliability

- *The system shall not crash the browser.*
- *If classification fails, a neutral message shall be displayed instead of incorrect results.*

Usability

- *Notifications shall be clearly visible within the Gmail interface.*
- *Messages shall use simple, non-technical language.*
- *No manual configuration shall be required for basic operation.*

Security and Privacy

- *Email content shall not be permanently stored.*
- *The system shall not transmit email content to unauthorized external systems.*
- *All processing shall occur within the browser environment.*

Maintainability

- *Requirements shall be uniquely identifiable for future modification.*
- *The system architecture shall allow updates to the AI model without redesigning the entire extension.*

Availability

The Email Detection system operates as a Chrome Extension within the Google Chrome browser and depends on the availability of the Gmail web interface. The system does not run as a continuously operating server and does not require 24/7 infrastructure-level uptime.

The following availability factors apply:

Accessible Time

The system shall be available whenever:

- *The Google Chrome browser is running, and*
- *The extension is enabled, and*
- *The user is accessing the Gmail web interface.*

System availability is therefore dependent on the browser session and internet connectivity.

Recovery Time

If the classification process fails due to temporary processing error, the system shall display a neutral status message within 3 seconds.

If the extension encounters a runtime error, refreshing the Gmail page shall restore normal operation within 5 seconds.

Restart Time

If the browser is restarted, the system shall automatically become operational once:

- *Chrome is reopened,*
- *The extension is loaded,*
- *Gmail is accessed.*

Startup initialization time shall not exceed 5 seconds after the Gmail page loads.

Maintenance Time

The system does not require scheduled maintenance downtime. Updates to the extension may require:

- *Browser restart or extension reload.*

Update-related downtime shall not exceed the duration of browser restart.

Security

The Email Detection system shall implement security measures to protect user data from unauthorized access, modification, disclosure, or misuse. As the system operates within the browser environment, security considerations focus primarily on data privacy, local processing, and controlled internal access.

a) Cryptographical Techniques

The system does not implement custom cryptographic protocols. Communication between the browser and Gmail servers is secured through standard HTTPS encryption managed by the browser.

The extension shall not transmit email content to external servers for processing. All AI classification shall be performed locally within the browser environment.

b) Logging and History Data

The system shall not permanently store email content or classification results.

No persistent log files containing user email data shall be created. Temporary processing data shall be cleared after classification is completed.

c) Modular Function Assignment

The system shall separate functionality into logical components:

- *Content extraction module*
- *AI classification module*
- *Notification display module*

This separation reduces unintended data exposure between components.

d) Restricted Internal Communication

Email content shall only be accessible to the internal classification module. No external scripts or third-party modules shall have access to extracted email content.

The extension shall operate under Chrome Extension permission policies and request only minimal required permissions.

e) Data Integrity

The system shall process only visible email subject and body text extracted from the Gmail interface.

The system shall validate extracted text input before sending it to the AI classification module to prevent malformed or corrupted data from affecting system behavior.

Maintainability

The Email Detection system shall be designed to support maintenance activities during development and future updates.

The following maintenance activities are supported:

- ***Model Update:** The AI classification model can be updated or replaced without redesigning the entire extension structure.*
- ***Bug Fixing:** Code modifications can be applied to improve performance, classification accuracy, or interface behavior.*
- ***UI Improvements:** Notification design and message wording can be adjusted without affecting core classification logic.*
- ***Compatibility Updates:** The system may require updates if significant structural changes occur in the Gmail web interface.*

During the project period, maintenance includes testing, debugging, and refinement.
No formal warranty service agreement applies, as the system is developed as an academic project.

The modular design (content extraction, classification, notification) supports easier modification and maintenance.

Portability

The Email Detection system is designed to operate in the following environment:

- ***Operating Systems:***
  - *Windows 10 or later*
  - *macOS (latest supported versions)*
  - *Linux distributions capable of running Google Chrome*
- ***Web Browser:***
  - *Google Chrome (latest stable version recommended)*

The system is not designed for:

- *Mobile devices*

- *Chrome mobile browser*
- *Other browsers (e.g., Firefox, Safari, Edge) unless manually adapted*

Expected Performance:

- *Classification result displayed within 3 seconds under normal desktop operating conditions.*
- *Performance may vary depending on CPU capability and available memory.*

The system is portable across desktop operating systems that support Google Chrome but is not cross-browser portable without modification.

Other Requirements

Operations

The Email Detection system operates as a browser-based extension and does not require complex operational procedures. The following operational aspects apply:

a) Modes of Operation

The system operates in a single mode:

- ***Active Monitoring Mode:***
  When a user opens an email in the Gmail web interface, the system automatically analyzes the content and displays a classification result.

There are no separate operational modes such as offline mode, batch processing mode, or administrative mode.

b) Project Timeline and Calendar

The project development follows an academic development schedule, including:

- *Requirement analysis*
- *System design*
- *AI model training*
- *Implementation of Chrome Extension*
- *Testing and refinement*
- *Documentation preparation*

No ongoing operational calendar is required after deployment, as the system runs automatically when installed.

c) Data Processing Support Functions

The system performs basic internal data processing functions, including:

- *Text extraction from opened emails*
- *Text preprocessing before classification*
- *Passing structured input to the AI model*

The system does not require external data cleansing, data conversion services, or database preprocessing operations.

d) Backup and Recovery Operations

The system does not maintain persistent data storage. Therefore:

- *No backup system is required.*
- *No database recovery procedures are necessary.*

If the extension fails, reloading the browser page restores functionality.

Site Adaptation

The Email Detection system does not require site-specific configuration or installation customization.

There are:

- *No environment-specific grid values*
- *No safety limits*
- *No mission-based parameter adjustments*

The system functions identically on any supported desktop environment running Google Chrome and accessing Gmail.

No site-specific initialization sequences are required.

Appendixes

The appendixes included in this document provide supporting information for better understanding of the Email Detection system. These appendixes are provided for reference purposes only and are ***not considered mandatory software requirements*** *unless explicitly stated.*

Appendix A: Sample Input and Output

This appendix provides examples of email input content and corresponding system output.

Sample Input (Email Content)

Subject:
"Your Account Has Been Suspended"

Body:
"Click here immediately to verify your account information…"

Sample Output

Classification: *Malicious*
Notification:
" Warning: This email may contain phishing content. Please verify the sender before taking any action."

Sample Subscription Email

Subject:
"Your Monthly Subscription Receipt"

Output:
Classification: Subscription-related
Notification:
"This email appears to be related to a subscription service. Please review the billing details and manage your subscription if necessary."

Appendix B: Problem Description

Email users frequently encounter phishing attempts, scam messages, and subscription-related notifications that may lead to financial risk or confusion. The software addresses this problem by providing AI-assisted classification and user notifications within the Gmail interface.

Appendix C: System Packaging

The system is distributed as a Google Chrome Extension package. Installation requires enabling the extension in the Chrome browser. No special export controls, encryption packaging requirements, or secure deployment procedures are required.

Statement on Appendix Status

The appendixes included in this document serve as supporting material and explanatory references. They are not binding functional requirements unless explicitly referenced within Section 3.

1. *Annex 1: Characteristics of a good SRS*

An SRS should be

a) Correct: An SRS is correct if, and only if, every requirement stated therein is one that the software shall meet.

b) Unambiguous: An SRS is unambiguous if, and only if, every requirement stated therein has only one interpretation. As a minimum, this requires that each characteristic of the final

product be described using a single unique term.

c) Complete: An SRS is complete if, and only if, it includes the following elements:

c.1) All significant requirements, whether relating to functionality, performance, design constraints, attributes, or external interfaces. In particular any external requirements imposed by a system specification should be acknowledged and treated.

c.2) Definition of the responses of the software to all realizable classes of input data in all realizable classes of situations. Note that it is important to specify the responses to both valid and invalid input values.

c.3) Full labels and references to all figures, tables, and diagrams in the SRS and definition of all terms and units of measure.

d) Consistent: Consistency refers to internal consistency. (Note: If an SRS does not agree with some higher-level document, such as a system requirements specification, then it is not correct). An SRS is internally consistent if, and only if, no subset of individual requirements described in it conflict.

e) Ranked for importance and/or stability: all of the requirements that relate to a software product are not equally important. Some requirements may be essential, especially for life-critical applications, while others may be desirable. Each requirement in the SRS should be identified to make these differences clear and explicit.

e.1) Degree of necessity:

- Essential: Implies that the software will not be acceptable unless these requirements are provided in an agreed manner.

- Conditional: Implies that these are requirements that would enhance the software product, but would not make it unacceptable if they are absent.

- Optional: Implies a class of functions that may or may not be worthwhile. This gives the supplier the opportunity to propose something that exceeds the SRS.

e.2) Degree of stability: One method of identifying requirements uses the dimension of stability. Stability can be expressed in terms of the number of expected changes to any requirement based on experience or knowledge of forthcoming events that affect the organization, functions, and people supported by the software system.

f) Verifiable: An SRS is verifiable if, and only if, every requirement stated therein is verifiable. A requirement is verifiable if, and only if, there exists some finite cost-effective process with which a person or machine can check that the software product meets the requirement. In general any ambiguous requirement is not verifiable. Nonverifiable requirements include statements such as "works well", "good human interface", or "shall usually happen".

If a method cannot be devised to determine whether the software meets a particular requirement, then that requirement should be removed or revised.

g) Modifiable: An SRS is modifiable if, and only if, its structure and style are such that any changes to the requirements can be made easily, completely, and consistently while retaining the structure and style. Modifiability generally requires an SRS to:

g.1) Have a coherent and easy-to-use organization with a table of contents, an index, and explicit crossreferencing;

g.2) Not be redundant (i.e., the same requirement should not appear in more than one place in the SRS);

g.3) Express each requirement separately, rather than intermixed with other requirements.

h) Traceable: An SRS is traceable if the origin of each of its requirements is clear and if it facilitates the referencing of each requirement in future development or enhancement

documentation. The following two types of traceability

are recommended:

h.1) Backward traceability (i.e., to previous stages of development): This depends upon each requirement

explicitly referencing its source in earlier documents.

h.2) Forward traceability (i.e., to all documents spawned by the SRS): This depends upon each requirement

in the SRS having a unique name or reference number. The forward traceability of the SRS is especially important when the software product enters the operation and maintenance phase. As code and design documents are modified, it is essential to be able to ascertain the complete set of requirements that may be affected by those modifications.

a) Correct Every requirement stated in this SDS reflects what the system actually implements. The Chrome Extension reads email text from Gmail DOM and sends it to the HuggingFace Gradio API, which returns a classification result of NOT SPAM, SPAM, or SUBSCRIPTION. All stated requirements are met by the delivered system.

b) Unambiguous Each requirement uses a single, consistent term throughout the document. For example, "email text" always refers to the plain text extracted from the Gmail DOM, "confidence" always refers to the percentage score returned by the model, and "Space" always refers to the HuggingFace hosted Gradio application. No term carries more than one meaning.

c) Complete

- **c.1)** All significant requirements are covered — functionality (scan, classify, display result), performance constraints (256-token limit, 2000-character truncation), design constraints (Chrome MV3, free-tier HuggingFace), and external interfaces (Gradio API, Gmail DOM).
- **c.2)** The system specifies responses to both valid inputs (English email text → classification result) and invalid inputs (Thai language → warning message, no email open → error message, API unavailable → connection error message).
- **c.3)** All terms are defined in Section 1.3 (Definitions, Acronyms and Abbreviations). The UML Deployment Diagram in Section 3 is labeled and referenced in the body text.

d) Consistent No conflicting requirements exist within this document. The technology constraints in Section 2.2 are consistent with the architecture described in Section 4. The Gradio API versioning constraint (Section 2.2.4) is directly reflected in the fallback mechanism described in Section 4.2. No requirement contradicts another.

e) Ranked for Importance and/or Stability

- **Essential:** Email text extraction from Gmail DOM, API call to HuggingFace Space, display of classification label (NOT SPAM / SPAM / SUBSCRIPTION).
- **Conditional:** Confidence percentage display, Detail section with reason or cancel steps, Thai language detection warning.
- **Optional:** Rescan button label change, loading spinner animation, email subject inclusion in scan payload.

In terms of stability, the core classification requirement is highly stable as it depends on the trained model. The API endpoint handling is moderately stable as it depends on Gradio

versioning which may change over time.

f) Verifiable Every requirement in this document is verifiable through direct testing. For example: "The extension shall display NOT SPAM, SPAM, or SUBSCRIPTION as the classification result" can be verified by scanning known emails and checking the output. "The extension shall display a warning when Thai text exceeds 30%" can be verified by scanning a Thai-language email and confirming the warning card appears. No vague statements such as "works well" or "good interface" are used.

g) Modifiable

- **g.1)** The document contains a clear section hierarchy (Introduction → Design Considerations → System Environment → Architecture → UX/UI Design), making it easy to locate and update specific sections.
- **g.2)** Each requirement appears only once. For example, the Gradio API fallback mechanism is described solely in Section 4.2 and referenced from Section 2.2.4 without duplication.
- **g.3)** Each requirement is expressed as a separate, standalone statement. The Thai language detection requirement, the token limit constraint, and the HuggingFace sleep policy are each written independently and do not intermix with one another.

h) Traceable

- **h.1) Backward traceability:** Requirements trace back to the original model training notebook (`Spam_Mail_Detector_BERT.ipynb`). For example, the 3-class label requirement (NOT SPAM, SPAM, SUBSCRIPTION) traces to the label mapping defined in Cell 4 of the notebook. The 256-token truncation constraint traces to the `max_length=256` parameter in Cell 8.
- **h.2) Forward traceability:** Each major requirement is identified by section number (e.g., Section 2.2.4 for Gradio versioning, Section 4.2 for the Chrome Extension component). These references allow any future modification — such as updating the HuggingFace Space URL or retraining the model with new labels — to be traced back to the specific sections and components that would be affected.

2. *Annex 2: Organizing the specific requirements*

For anything but trivial systems the detailed requirements tend to be extensive. For this reason, it is recommended that careful consideration be given to organizing these in a manner optimal for understanding. There is no one optimal organization for all systems. Different classes of systems lend themselves to different organizations of requirements in Section 3 of the SRS. Some of these organizations are:

a) System mode

Some systems behave quite differently depending on the mode of operation. For example, a control system may have different sets of functions depending on its mode: training, normal, or emergency. When organizing this section by mode, the outline in Annex 5.3.1. The choice depends on whether interfaces and performance are dependent on mode.

b) User class

Some systems provide different sets of functions to different classes of users. For example, an elevator control system presents different capabilities to passengers, maintenance workers, and fire fighters. When organizing this section by user class, the outline in Annex 5.3.2 should be used.

c) Objects

Objects are real-world entities that have a counterpart within the system. For example, in a

patient monitoring system, objects include patients, sensors, nurses, rooms, physicians, medicines, etc. Associated with each object is a set of attributes (of that object) and functions (performed by that object). These functions are also called services, methods, or processes. When organizing this section by object, the outline in Annex 5.3.3 should be used. Note that sets of objects may share attributes and services. These are grouped together as classes.

d) Feature

A feature is an externally desired service by the system that may require a sequence of inputs to effect the desired result. For example, in a telephone system, features include local call, call forwarding, and conference call. Each feature is generally described in a sequence of stimulus-response pairs. When organizing this section by feature, the outline in Annex 5.3.4 should be used.

e) Stimulus

Some systems can be best organized by describing their functions in terms of stimuli. For example, the functions of an automatic aircraft landing system may be organized into sections for loss of power, wind shear, sudden change in roll, vertical velocity excessive, etc. When organizing this section by stimulus, the outline in Annex 5.3.5 should be used.

f) Response

Some systems can be best organized by describing all the functions in support of the generation of a response. For example, the functions of a personnel system may be organized into sections corresponding to all functions associated with generating paychecks, all functions associated with generating a current list of employees, etc. The outline in Annex 5.3.5 (with all occurrences of stimulus replaced with response) should be used.

g) Functional hierarchy

When none of the above organizational schemes prove helpful, the overall functionality can be organized into a hierarchy of functions organized by either common inputs, common outputs, or common internal data access. Data flow diagrams and data dictionaries can be used to show the relationships between and among the functions and data. When organizing this section by functional hierarchy, the outline in Annex 5.3.6 should be used.

Additional comments:

Whenever a new SRS is contemplated, more than one of the organizational techniques given here may be appropriate. In such cases, organize the specific requirements for multiple hierarchies tailored to the specific needs of the system under specification. For example, see Annex 5.3.7 for an organization combining user class and feature. Any additional requirements may be put in a separate section at the end of the SRS.

There are many notations, methods, and automated support tools available to aid in the documentation of requirements. For the most part, their usefulness is a function of organization. For example, when organizing by mode, finite state machines or state charts may prove helpful; when organizing by object, object-oriented analysis may prove helpful; when organizing by feature, stimulus-response sequences may prove helpful; and when organizing by functional hierarchy, data flow diagrams and data dictionaries may prove helpful.

In any of the outlines given in Annex 5.3.1 through Annex 5.3.7, those sections called "Functional Requirement i" may be described in native language (e.g., English), in pseudocode, in a system definition language, or in four subsections titled: Introduction, Inputs, Processing, and Outputs.

The specific requirements in Section 3 are organized primarily using the ***Feature-based organization approach****, as recommended in Annex 2 of the IEEE guideline.*

Chosen Organization: Feature-Based Structure

The Email Detection system is best organized by features because:

- *The system provides clearly defined external services to the user.*
- *Each feature represents a distinct observable function.*
- *The system behavior follows a stimulus-response pattern.*

Each feature corresponds to an externally visible service, such as:

- *Email detection*
- *Email classification*
- *Malicious email notification*
- *Subscription guidance display*
- *Safe status display*

Each feature is described in terms of:

- *Input (stimulus)*
- *Processing*
- *Output (response)*

This structure improves clarity and readability.

Secondary Structure: Stimulus–Response Pattern

Within each feature, requirements follow a stimulus–response format:

Stimulus: *User opens an email*
Processing: *Extract text → AI classification*
Response: *Display classification result*

This approach ensures that all requirements are externally observable and verifiable.

Justification for Not Using Other Organizational Schemes

- ***System Mode:*** *Not applicable. The system operates in a single mode only.*
- ***User Class:*** *Not applicable. There is only one user type (Gmail user).*
- ***Object-Based Organization:*** *Not suitable, as the system does not model complex object entities.*

- ***Response-Based Only:*** *Less suitable, as multiple responses stem from a common classification feature.*
- ***Functional Hierarchy:*** *Not necessary due to limited system complexity.*

Documentation Style

Each functional requirement in Section 3 is written in structured natural language and includes:

- *Introduction/Description*
- *Input (Stimulus)*
- *Processing*
- *Output (Response)*

This structure aligns with the IEEE recommendations for organizing non-trivial systems.

Conclusion

The feature-based organization combined with stimulus–response formatting provides the clearest and most appropriate structure for the Email Detection system.

3. *Annex 3: SRS templates*

1. Template of SRS Section 3 organized by feature

3. Specific requirements
3.1 External interface requirements
3.1.1 User interfaces

The system provides a notification panel integrated within the Gmail web interface.

- *A status indicator appears when an email is opened.*
- *A warning message is displayed if the email is classified as malicious.*
- *A guidance message is displayed if the email is classified as subscription-related.*
- *A safe indicator is displayed if the email is classified as safe.*
- *If classification fails, a neutral message is shown.*

All UI elements are embedded within the Gmail webpage and do not modify email content.

3.1.3 Software interfaces

The system interfaces with:

- *Google Chrome browser (Extension environment)*

- *Gmail web interface (DOM interaction)*

The AI model is integrated locally within the extension and does not use external APIs.

3.1.4 Communications interfaces

The system does not implement custom communication protocols.

The Gmail platform communicates over HTTPS, managed by the browser.
The extension itself does not transmit email data to external servers.

3.2 System features
3.2.1 System Feature 1 :Email Detection and Classification
3.2.1.1 Introduction/Purpose of feature

This feature enables automatic detection and classification of an opened email within the Gmail interface. The purpose is to assist users in identifying whether an email is safe, malicious, or subscription-related.

3.2.1.2 Stimulus/Response sequence

Stimulus:
User opens an email in Gmail.

System Processing:

1. *Detect opened email.*
2. *Extract subject and body text.*
3. *Send extracted text to AI classification module.*

Response:
Display classification result within the Gmail interface.

3.2.1.3 Associated functional requirements
3.2.1.3.1 Functional requirement 1: The system shall detect when a user opens an email in the Gmail web interface.
3.2.1.3.2 Functional requirement 2: The system shall extract visible subject and body text from the opened email.
3.2.1.3.3 Functional requirement 3: The system shall classify extracted content into one of the following categories:

- *Safe*
- *Malicious*
- *Subscription-related*

3.2.1.3.4 Functional requirement 4: If classified as malicious, the system shall display a visible warning notification.
3.2.1.3.5 Functional requirement 5: If classified as subscription-related, the system shall display AI-generated guidance suggesting subscription management steps.
3.2.1.3.6 Functional requirement 6: If classified as safe, the system shall display a safe

status indicator.
3.2.1.3.7 Functional requirement 7: The system shall not modify, delete, or automatically act on any email content.
3.2.2 System feature 2

Stimulus / Response Sequence

Stimulus:
AI classification result = “Malicious”

System Processing:

1. *Generate warning message.*
2. *Apply warning styling (e.g., red indicator).*

Response:
Display a visible warning panel above or within the email content area.

3.2.2.3 Associated Functional Requirements

3.2.2.3.1 Functional Requirement 1

The system shall display a warning notification when an email is classified as malicious.

3.2.2.3.2 Functional Requirement 2

The warning notification shall be visually distinguishable from normal messages.

3.2.2.3.3 Functional Requirement 3

The system shall not automatically delete or modify the email.

3.2.3 System Feature 3: Subscription Guidance Assistance

3.2.3.1 Introduction / Purpose of Feature

This feature provides AI-generated guidance when an email is classified as subscription-related.
The purpose is to help users manage subscription services effectively.

3.2.3.2 Stimulus / Response Sequence

Stimulus:
AI classification result = “Subscription-related”

System Processing:

1. *Identify subscription-related keywords.*
2. *Generate management guidance message.*

Response:
Display guidance text within the Gmail interface.

3.2.3.3 Associated Functional Requirements

3.2.3.3.1 Functional Requirement 1

The system shall display subscription management guidance when an email is classified as subscription-related.

3.2.3.3.2 Functional Requirement 2

The guidance shall be displayed without altering the original email content.

3.2.3.3.3 Functional Requirement 3

The system shall allow the user to ignore or dismiss the guidance message.

3.2.4 System Feature 4: Safe Email Status Display

3.2.4.1 Introduction / Purpose of Feature

This feature informs users when an email is classified as safe.
The purpose is to provide reassurance and transparency of classification results.

3.2.4.2 Stimulus / Response Sequence

Stimulus:
AI classification result = “Safe”

System Processing:
Generate safe status indicator.

Response:
Display safe confirmation message within the interface.

3.2.4.3 Associated Functional Requirements

3.2.4.3.1 Functional Requirement 1

The system shall display a safe status indicator when an email is classified as safe.

3.2.4.3.2 Functional Requirement 2

The safe indicator shall be visually different from malicious warning messages.

3.2.5 System Feature m: System Error Handling

3.2.5.1 Introduction / Purpose of Feature

This feature ensures graceful handling of classification errors or unexpected system behavior.

3.2.5.2 Stimulus / Response Sequence

Stimulus:

AI model fails to classify email or processing error occurs.

System Processing:

Generate fallback message.

Response:

Display neutral status message to the user.

3.2.5.3 Associated Functional Requirements

3.2.5.3.1 Functional Requirement 1

The system shall display a neutral message when classification fails.

3.2.5.3.2 Functional Requirement 2

The system shall not crash or freeze during classification failure.

3.3 Performance requirements

- *The system shall complete email classification within **2 seconds** after an email is opened under normal operating conditions.*
- *The system shall not cause Gmail page loading delay exceeding **1 second**.*
- *The system shall operate with average CPU usage not exceeding **20%** during classification.*
- *The system shall operate within **200 MB RAM usage** under normal conditions.*
- *The system shall support processing of emails up to **50,000 characters** in length.*
- *The system shall maintain stable performance for continuous usage of at least **4 hours** without requiring restart.*

3.4 Design constraints

*The system shall be implemented as a **Google Chrome Extension**.*

*The system shall comply with **Chrome Extension Manifest V3** requirements.*

The system shall operate only within the Gmail web interface.

The system shall not modify Gmail backend services.

The system shall not transmit email data to external servers (local processing only).

The system shall be developed using:

- *JavaScript*
- *HTML*
- *CSS*

The AI model must be lightweight enough to run within browser environment constraints.

3.5 Software system attributes

Reliability

- *The system shall operate without unexpected crashes.*
- *The system shall recover automatically if classification fails.*
- *The system shall not corrupt email data.*

Availability

- *The system shall be available whenever Google Chrome and Gmail are accessible.*
- *Recovery time after minor error shall not exceed **5 seconds**.*
- *Restart of the extension shall not exceed **10 seconds**.*

Security

- *The system shall not store email content permanently.*
- *The system shall not transmit email data externally.*
- *The system shall not request unnecessary browser permissions.*
- *The system shall restrict access to classification logic within extension scope.*

Maintainability

- *The system code shall be modular (separate UI, classification, and detection modules).*
- *The system shall allow AI model updates without full architecture redesign.*
- *Bug fixes shall not require modification of Gmail interface.*

Portability

- *The system shall operate on:*

- *Windows 10+*
- *macOS*
- *Linux*

- *The system shall operate on:*
  - *Google Chrome version 110 or higher.*
- *Expected performance degradation shall not exceed 10% across supported platforms.*

3.6 Other requirements

Operational Requirements

- *The system shall operate during normal Gmail usage.*
- *No special training is required for users.*
- *The system shall not interrupt user workflow.*

Backup and Recovery

- *The system does not store persistent user data.*
- *No backup procedure is required.*
- *In case of malfunction, reinstalling the extension restores functionality.*

Site Adaptation

- *No site-specific configuration is required.*
- *The system automatically adapts to the Gmail interface layout.*

2. **Template of SRS Section 3 organized by user class**

3. *Specific requirements*

3.1 External interface requirements

3.1.1 User interfaces

The system provides a single popup interface accessible via the Chrome Extension icon in the browser toolbar. The popup consists of the following elements:

- **Header** — displays the application name "Maily Detector" and subtitle "Chrome Extension" on a dark background (`#1a1a2e`).
- **Scan Button** — a circular green button centered in the popup. On first use the label reads "Scan". After the first successful scan the label permanently changes to "Rescan" for the remainder of the session.
- **Status Card** — appears after scanning. Displays the classification label (NOT SPAM / SPAM / SUBSCRIPTION) in a color-coded format (green / red / blue respectively), accompanied by a confidence percentage bar.
- **Detail Card** — appears below the Status Card. For SPAM and NOT SPAM, displays a plain-text reason. For SUBSCRIPTION, displays a numbered step-by-step guide for

cancelling a subscription.
- **Warning Card** — appears when the scanned email contains more than 30% Thai-language characters. Informs the user that results may be inaccurate and provides a "Scan Anyway" option.
- **Error Card** — appears when the extension cannot read the email or cannot connect to the API. Displays a human-readable error message.

The interface does not require any user login or configuration. No settings panel or options page is provided.

3.1.2 Hardware interfaces

The system does not interact with any hardware devices directly. The minimum hardware requirement is any personal computer or laptop capable of running Google Chrome version 88 or later. No special peripherals, sensors, or hardware-specific APIs are used.

3.1.3 Software interfaces

Software	Version	Interface Type	Purpose
Google Chrome	88+	Browser Runtime	Hosts the extension and provides Chrome Extension APIs
Gmail	Web (mail.google.com)	Web DOM	Source of email text extracted by content.js
HuggingFace Spaces	Gradio 4.x / 3.x	REST API (HTTPS)	Hosts the DistilBERT inference model
Chrome Extension API	Manifest V3	JavaScript API	Enables message passing between content.js and popup.js
Gradio API	4.x (fallback 3.x)	HTTP POST / SSE	Receives email text and returns classification result

3.1.4 Communications interfaces

- **Protocol:** HTTPS (TLS encrypted) for all communication between the extension and HuggingFace Space.
- **Request format:** JSON payload — `{ "data": ["<email_text>"] }` sent via HTTP POST.
- **Response format (Gradio 4.x):** Two-step — POST returns an `event_id`, followed by a GET request that streams the result via Server-Sent Events (SSE).
- **Response format (Gradio 3.x fallback):** Single POST returns a synchronous JSON response — `{ "data": ["<result_string>"] }`.
- **Internal communication:** Chrome message passing API (`chrome.tabs.sendMessage`) is used between popup.js and content.js. No network is involved in this internal channel.

3.2 Functional requirements

3.2.1 User class 1

The only user class in this system is the General User: a person who has installed the Chrome Extension and uses Gmail to read email. No authentication, registration, or role management is required.

3.2.1.1 Email Text Extraction The system shall automatically extract the subject line and body text of the currently open email in Gmail when the user triggers a scan. Extraction shall use known Gmail DOM selectors. If no email is open, the system shall display an error message instructing the user to open an email first.

3.2.1.2 Language Detection The system shall calculate the ratio of Thai Unicode characters (U+0E00–U+0E7F) to total non-whitespace characters in the extracted email text. If this ratio exceeds 30%, the system shall display a warning card notifying the user that the model was trained on English data only and that results may be inaccurate. The user shall still be able to proceed with scanning by pressing "Scan Anyway".

3.2.1.3 API Request The system shall send the extracted email text (truncated to a maximum of 2,000 characters) to the HuggingFace Gradio inference endpoint via HTTPS POST. The system shall first attempt the Gradio 4.x endpoint (`/gradio_api/call/predict`). If this request fails, the system shall automatically retry using the Gradio 3.x endpoint (`/run/predict`).

3.2.1.4 Classification Result Display The system shall display the classification result returned by the API in the Status Card. The label shall be one of: NOT SPAM, SPAM, or SUBSCRIPTION. The label shall be displayed in its corresponding color (green, red, or blue). A confidence percentage bar shall reflect the actual confidence score returned by the model.

3.2.1.5 Detail Display The system shall display additional information in the Detail Card based on the classification result:

- NOT SPAM: a plain-text explanation stating no spam indicators were detected.
- SPAM: a plain-text explanation identifying the suspicious patterns detected.
- SUBSCRIPTION: a numbered 5-step guide describing how to cancel a subscription.

3.2.1.6 Rescan After the first scan is completed, the Scan button label shall change from "Scan" to "Rescan". Pressing the button again shall repeat the full scan process on the currently open email.

3.2.1.7 Error Handling The system shall display a descriptive error message in the following conditions:

- The user is not on a Gmail page.
- No email is currently open in Gmail.
- The extension cannot read the Gmail DOM (e.g., after a page refresh).
- The HuggingFace Space cannot be reached (network error or Space is sleeping).
- The API returns an unrecognizable response format.

3.3 Performance requirements

- **Response time:** The system shall return a classification result within 10 seconds under normal operating conditions (Space awake, stable internet connection). Typical response time is expected to be 2–8 seconds due to CPU-only inference on the HuggingFace free tier.
- **Input size:** The system shall truncate email text to a maximum of 2,000 characters before sending to the API, ensuring consistent payload size regardless of email length.

- **Token limit:** The inference model processes a maximum of 256 tokens per input. Text exceeding this limit is silently truncated by the tokenizer on the server side.
- **Concurrency:** The system is designed for single-user personal use only. No concurrent request handling or queuing mechanism is implemented.
- **Availability:** System availability depends on the HuggingFace Spaces free tier. The Space may enter a sleep state after approximately 30 minutes of inactivity and require a manual wake-up by visiting the Space URL before scanning.

3.4 Design constraints

- **Chrome Manifest V3:** The extension must conform to Chrome Extension Manifest Version 3. Background pages are not permitted; all persistent logic must use Service Workers. Cross-context communication must use the Chrome message passing API.
- **Gmail only:** The content script is restricted to `mail.google.com`. The extension does not function on any other email client or web application.
- **English language only:** The classification model was trained exclusively on English-language data. The system is not designed to classify non-English emails accurately.
- **No local model:** The system does not bundle or run any machine learning model locally in the browser. All inference is performed remotely via the HuggingFace Space API.
- **No data persistence:** The system shall not store, log, or transmit any email content beyond the single API call required for classification. No user data is retained between sessions.
- **Unpacked extension only:** The extension is distributed as an unpacked developer extension and is not published on the Chrome Web Store. Installation requires Developer Mode to be enabled in Chrome.

3.5 Software system attributes

Reliability The system implements a two-endpoint fallback mechanism (Gradio 4.x → Gradio 3.x) to maintain functionality across Gradio version changes. All error conditions are handled gracefully with descriptive user-facing messages rather than silent failures.

Security All communication with the HuggingFace Space is conducted over HTTPS. The extension does not collect, store, or transmit any personally identifiable information. Email text is sent to the API solely for classification and is not logged or retained by the extension. The extension requests only the minimum permissions required: `activeTab`, `scripting`, and access to `mail.google.com` and `*.hf.space`.

Maintainability The extension code is separated into four files with distinct responsibilities: `manifest.json` (configuration), `content.js` (DOM extraction), `popup.html` (UI structure), and `popup.js` (API logic and rendering). This separation allows any single component to be modified without affecting the others. The HuggingFace Space URL is defined as a single constant at the top of `popup.js`, making it easy to update if the Space is moved or renamed.

Usability The interface follows Hick's Law by presenting only one primary action at any time. Color coding (green/red/blue) provides immediate visual feedback consistent with standard UI color conventions. Error and warning messages are written in plain English without technical jargon.

Portability The extension is compatible with any operating system that supports Google Chrome version 88 or later, including Windows, macOS, and Linux. No platform-specific dependencies are introduced.

3.6 Other requirements

- The HuggingFace Space (`meowslouise/Spammail-detector`) must be manually kept active. If the Space is deleted or made private, the extension will cease to function.
- The extension requires an active internet connection at all times during use. Offline functionality is not supported.
- Future retraining of the model with additional languages or classes would require updates to both the Space deployment and the `STATUS_CONFIG` and `parseGradioString()` logic in `popup.js`.

3. Template of SRS Section 3 showing multiple organizations

3. *Specific requirements*

3.1 External interface requirements

3.1.1 User interfaces

The system provides a single popup interface accessible via the Chrome Extension icon in the browser toolbar. The popup consists of the following elements:

- **Header** — displays "Maily Detector" and subtitle "Chrome Extension" on a dark navy background.
- **Scan Button** — a circular green button. Label reads "Scan" on first open and changes to "Rescan" after the first completed scan.
- **Status Card** — displays the classification label (NOT SPAM / SPAM / SUBSCRIPTION) with color coding and a confidence percentage bar.
- **Detail Card** — displays a reason for SPAM/NOT SPAM, or a 5-step cancellation guide for SUBSCRIPTION.
- **Warning Card** — appears when Thai text exceeds 30% of the email content.
- **Error Card** — appears when the email cannot be read or the API cannot be reached.

3.1.2 Hardware interfaces

The system does not interact with any hardware devices directly. The minimum requirement is any personal computer capable of running Google Chrome version 88 or later. No special peripherals or hardware-specific APIs are used.

3.1.3 Software interfaces

Software	Version	Interface Type	Purpose
Google Chrome	88+	Browser Runtime	Hosts the extension and provides Chrome Extension APIs
Gmail	Web (mail.google.com)	Web DOM	Source of email text extracted by content.js
HuggingFace Spaces	Gradio 4.x / 3.x	REST API (HTTPS)	Hosts the DistilBERT+LoRA inference model
Chrome Extension API	Manifest V3	JavaScript API	Enables message passing between content.js and popup.js

3.1.4 Communications interfaces

- **Protocol:** HTTPS for all communication between the extension and HuggingFace Space.
- **Request format:** JSON payload — `{ "data": ["<email_text>"] }` via HTTP POST.
- **Response format (Gradio 4.x):** POST returns an `event_id`, followed by a GET request streaming the result via Server-Sent Events (SSE).
- **Response format (Gradio 3.x fallback):** Single POST returns synchronous JSON — `{ "data": ["<result_string>"] }`.
- **Internal communication:** Chrome message passing API (`chrome.tabs.sendMessage`) between popup.js and content.js.

3.2 Functional requirements

3.2.1 User class 1— General User

The only user class in this system is the General User: a person who has installed the Maily Detector Chrome Extension and uses Gmail to read email. No authentication or role management is required.

3.2.1.1 Feature 1.1

3.2.1.1.1 Introduction/Purpose of feature

This feature is the core function of the system. It allows the user to trigger an AI-powered classification of the currently open email in Gmail. The purpose is to help the user quickly identify whether an email is safe, spam, or subscription-related without manually reading the full content.

3.2.1.1.2 Stimulus/Response sequence

Step	Stimulus	Response
1	User opens an email in Gmail	Gmail renders the email in the DOM
2	User clicks the extension icon	Extension popup opens showing the Scan button
3	User clicks "Scan"	Button label changes to "..." and spinner appears
4	content.js extracts email subject and body from DOM	Email text (max 2,000 characters) is passed to popup.js
5	popup.js sends POST request to HuggingFace Space API	API processes the text through DistilBERT+LoRA model
6	API returns classification result	popup.js parses label, confidence, and reason from response
7	Result is rendered	Status Card and Detail Card appear; button changes to "Rescan"

3.2.1.1.3 Associated functional requirements

- The system shall extract the email subject and body text from the currently open Gmail email using DOM selectors `.a3s.aiL`, `.ii.gt .a3s`, and related fallback selectors.
- The system shall truncate extracted text to a maximum of 2,000 characters before transmission.
- The system shall first attempt the Gradio 4.x endpoint (`/gradio_api/call/predict`) and automatically fall back to the Gradio 3.x endpoint (`/run/predict`) if the first request fails.
- The system shall parse the classification label from the API response string, extracting exactly one of: NOT SPAM, SPAM, or SUBSCRIPTION.
- The system shall parse the confidence percentage from the API response string using the pattern `(\d+(?:\.\d+)?)%`.
- The system shall display the classification label in its corresponding color: green (`#3B6D11`) for NOT SPAM, red (`#A32D2D`) for SPAM, and blue (`#185FA5`) for SUBSCRIPTION.
- The system shall display a confidence bar reflecting the actual percentage returned by the model. If no confidence value is present in the response, the bar shall display "N/A".

3.2.1.2 Feature 1.2— Thai Language Detection

3.2.1.2.1 Introduction/Purpose of feature

The classification model was trained exclusively on English-language data. This feature detects when an email contains a significant proportion of Thai text and warns the user before scanning, preventing misleading results from being presented without context.

3.2.1.2.2 Stimulus/Response sequence

Step	Stimulus	Response
1	User clicks "Scan"	System extracts email text from DOM
2	System calculates Thai character ratio	If ratio > 30%, scanning is paused
3	Warning Card is displayed	User sees the Thai percentage and a "Scan Anyway" button
4a	User clicks "Scan Anyway"	System proceeds with the API call and displays result normally
4b	User closes the popup	No scan is performed

3.2.1.2.3 Associated functional requirements

- The system shall calculate the ratio of Thai Unicode characters (U+0E00–U+0E7F) to total non-whitespace characters in the extracted email text.
- If the Thai ratio exceeds 0.30 (30%), the system shall pause the scan and display the Warning Card before making any API call.
- The Warning Card shall display the detected Thai percentage (e.g., "This email contains 45% Thai text").

- The Warning Card shall state that the model was trained on English data only and that results may be inaccurate.
- A "Scan Anyway" button shall be presented, allowing the user to override the warning and proceed with classification.

3.2.1.3 Feature 1.3 — Detail Display

3.2.1.3.1 Introduction/Purpose of Feature

After classification, the user needs actionable context beyond just a label. This feature provides a human-readable explanation for SPAM and NOT SPAM results, and a practical step-by-step cancellation guide for SUBSCRIPTION results, helping the user decide what to do next.

3.2.1.3.2 Stimulus/Response Sequence

Step	Stimulus	Response
1	**API returns classification result**	**System determines the label**
2a	**Label is SPAM**	**Detail Card displays reason from API (e.g., "Contains suspicious keywords: free, win, click")**
2b	**Label is NOT SPAM**	**Detail Card displays confirmation that no spam indicators were detected**
2c	**Label is SUBSCRIPTION**	**Detail Card displays a 5-step numbered guide for cancelling a subscription**

3.2.1.3.3 Associated Functional Requirements

- The system shall extract the reason text from the API response string using the pattern `REASON\s*[:\-]\s*(.+?)(?:\n|$)`.
- For SPAM and NOT SPAM, the system shall display the extracted reason as plain text in the Detail Card. If no reason is present in the response, a predefined fallback message shall be used.
- For SUBSCRIPTION, the system shall always display the following fixed 5-step guide regardless of the API reason field:
    1. Log in to your account
    2. Go to Settings → Billing
    3. Click "Cancel Subscription"
    4. Confirm cancellation
    5. Check your email for confirmation

- The Detail Card section label shall read "DETAIL" in uppercase.

3.2.1.4 Feature 1.4 — Error Handling

3.2.1.4.1 Introduction/Purpose of Feature

The system operates across multiple external dependencies (Gmail DOM, HuggingFace Space, network connection). This feature ensures that all failure conditions are communicated clearly to the user rather than resulting in silent failures or unhandled exceptions.

3.2.1.4.2 Stimulus/Response Sequence

Step	Stimulus	Response
1	User clicks "Scan" while not on Gmail	Error Card: "Please open Gmail and click an email before scanning."
2	User clicks "Scan" with no email open	Error Card: "No email found. Click on an email first."
3	content.js cannot communicate with popup.js	Error Card: "Could not read email. Try refreshing Gmail and scan again."
4	HuggingFace Space is sleeping or unreachable	Error Card: "Cannot connect to HuggingFace Space. Make sure the Space is running."
5	API returns unrecognizable response format	Error Card displays the specific error message returned

3.2.1.4.3 Associated Functional Requirements

- The system shall verify that the active tab URL contains `mail.google.com` before attempting any DOM extraction.
- The system shall verify that content.js returns a non-empty text response before making any API call.
- The system shall wrap all API calls in try/catch blocks. Network errors shall display a connection error message. Unexpected response formats shall display the raw error message.
- All error messages shall be displayed in the Error Card with a red border and a ❌ icon.
- All error states shall be clearable by performing a new scan.

3.3 Performance requirements

- **Response time:** The system shall return and display a classification result within 10 seconds under normal conditions (Space awake, stable internet). Typical response time is 2–8 seconds due to CPU-only inference on the HuggingFace free tier.
- **Input size:** Email text shall be truncated to 2,000 characters maximum before transmission to ensure consistent API payload size.
- **Token limit:** The model processes a maximum of 256 tokens. Text exceeding this is truncated server-side by the tokenizer.
- **Concurrency:** The system is designed for single-user personal use. No concurrent request handling is implemented.

- **Availability:** System availability depends on the HuggingFace Spaces free tier. The Space may sleep after approximately 30 minutes of inactivity and requires a manual wake-up before use.

3.4 Design constraints

- **Chrome Manifest V3:** The extension must conform to MV3. All cross-context communication must use the Chrome message passing API (`chrome.tabs.sendMessage`).
- **Gmail only:** The content script is restricted to `mail.google.com`. The extension does not function on any other email client.
- **English language only:** The model was trained exclusively on English data. Accurate classification of non-English emails is not guaranteed.
- **No local model:** All inference is performed remotely via the HuggingFace Space. No ML model is bundled within the extension.
- **No data persistence:** The system shall not store, log, or transmit any email content beyond the single API call required for classification.
- **Unpacked extension only:** The extension is distributed as an unpacked developer extension and is not published on the Chrome Web Store.

3.5 Software system attributes

Reliability The system implements a two-endpoint fallback (Gradio 4.x → Gradio 3.x) to handle API version changes. All error conditions produce a descriptive user-facing message rather than a silent failure.

Security All communication uses HTTPS. The extension requests only minimum permissions: `activeTab`, `scripting`, and host access to `mail.google.com` and `*.hf.space`. No personally identifiable information is stored or retained by the extension.

Maintainability Code is separated into four files with distinct responsibilities: `manifest.json`, `content.js`, `popup.html`, and `popup.js`. The HuggingFace Space URL is defined as a single constant (`SPACE_BASE`) at the top of `popup.js`, allowing easy updates without modifying any other part of the codebase.

Usability The interface follows Hick's Law — only one primary action is presented at any time. Color coding provides immediate visual feedback. All messages use plain English without technical jargon, consistent with the Language in UX principle covered in course material.

Portability The extension is compatible with any OS supporting Chrome 88+, including Windows, macOS, and Linux.

3.6 Other requirements

- The HuggingFace Space (`meowslouise/Spammail-detector`) must remain active and publicly accessible. If the Space is deleted, made private, or the URL changes, the extension will cease to function until `SPACE_BASE` in `popup.js` is updated.
- An active internet connection is required at all times during use. Offline functionality is not supported.
- Future retraining of the model with additional languages or new classification labels would require updates to the `STATUS_CONFIG` object and `parseGradioString()` function in `popup.js`, as well as corresponding updates to the Detail display logic in

`renderDetail()`.

Annex 4: SRS Traceability Matrix

User (or feature)	**Refined list of functional or non-functional requirements**	**Link to a refined list of technical requirements**	**Priority (High, Medium, Low)**
e.g. Client	e.g. UC1: Buy a product: 1.1. Browse products 1.1.1. by category 1.1.2. by keyword(s) 1.2 Select product 1.3. Indicate amount 1.4. Check inventory 1.5. Select pay method 1.5.1. Process credit card payment 1.5.2. Process check payment 1.5.3. ….	e.g. Cookbooks C1, C2 C1.Recipe2.1: create list of items C1.Recipe2.1.1: sort list by a column C1.Recipe2.1.2: filter list by a column keyword C1.Recipe2.2: select row C1.Recipe2.2: enter valid number field C1.Recipe4.1: execute query and return evaluation of a condition C2.Recipe1.1: select from radio button or list C2.Recipe1.1.1: execute payment webservice C2.Recipe1.1.2: execute check web service …..	e.g. H H H M H H H H H L
….			